

Project Management Report

Jannis Arnold Vangelis Didaskalou Daniel Malik Aleksandar Manev
Jason Mann Aaron Miritz Christian Nix

February 13, 2026

Contents

1 Overall Project Vision	2
2 Project Timeline	2
3 System Architecture	3
3.1 Technology Stack	4
3.2 The KakoAI Toolbox	4
3.2.1 BOM Extraction Tool	4
3.2.2 Demand Analysis Tools	5
3.2.3 Procurement Tool	6
4 Method	6
4.1 Single-Agent vs. Multi-Agent System	6
4.2 DSPy: Programming over Prompting	7
4.3 Prompt Engineering	7
5 Team Chart	8
6 Current Progress and Future Plans	9
6.1 Status	9
6.2 Outlook	9
7 Acknowledgement of GenAI Usage	9
8 Pitch Video	9

1 Overall Project Vision

KakoAI is an industrial copilot designed to automate critical workflows for the cable confection company KAKO Elektro. The system acts as a secure bridge between unstructured customer data (technical drawings/PDFs) and structured enterprise resources (ERP systems, procurement APIs).

The core vision is to drastically reduce the manual workload of order approval. Currently, this process is a labor-intensive and cross-departmental effort that can take a long time to complete (see Figure 1). The process begins when a sales employee receives an email with a technical drawing of the ordered cable as a PDF. This drawing contains both the instructions for assembling the cable (only relevant once production begins) and a table with the required components and their quantities, known as the Bill of Materials (BOM). The employee must manually extract the BOM from the drawing into an Excel sheet, then cross-reference each component with the internal inventory system to find an internal part number (from the Xentral ERP system).

The sales employee then sends this Excel sheet to the warehouse team, who checks if the order can be fulfilled with the current inventory. This is currently done in two ways. First, they check the inventory management system (Xentral ERP) for the availability of the required components. However, given that the system is relatively new and employees (especially older staff) struggle to keep it up to date, they often also verify the physical warehouse stock manually. If the order cannot be fulfilled, the warehouse team searches for the missing parts on the procurement website Octopart, which provides real-time market data on electronic components, including price, lead times, and stock levels. They then order the missing components from the best supplier and report the availability or expected delivery date, allowing the sales team to confirm the order with the customer.

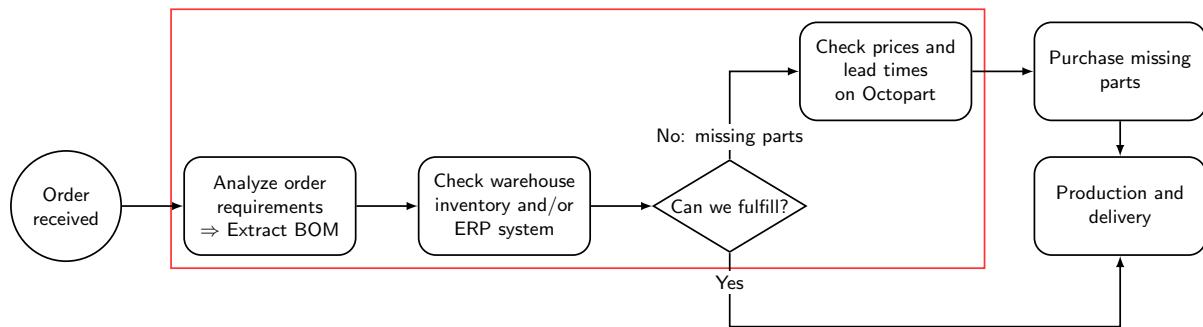


Figure 1: **Order Fulfillment Process Flowchart.** The current workflow is spread across multiple departments and involves manual data extraction, cross-referencing, and decision-making steps. The red box highlights the specific stages that KakoAI aims to automate.

Our vision for KakoAI is to automate this workflow, while keeping a human in the loop for any critical decisions, such as placing purchase orders. Specifically, we automatically extract the BOMs—**allowing users to verify and edit the results**—and cross-reference them with real-time inventory. Additionally, we make the Xentral ERP system accessible through a natural language interface, enabling less digitally native employees to easily use the system without needing to learn complex software. This lowers the barrier to entry and increases adoption across the company, leading to consistent and accurate inventory data. Finally, we streamline the procurement process for missing parts by integrating with the Octopart API to fetch real-time market data. KakoAI aims to transform a process that typically takes hours of manual cross-checking into a streamlined, semi-automated workflow.

Unlike generic chatbots, KakoAI is engineered as a domain-specific tool. It respects the *human-in-the-loop* principle, ensuring that while AI handles the heavy lifting of data extraction and market analysis, the final validation always rests with the engineer.

2 Project Timeline

Our project timeline is visualized as a Gantt chart in Figure 2, outlining the key phases and milestones throughout the project duration. We planned to have some buffer time for unexpected challenges from

the beginning, which allowed us to stay on track and finish the project. The presented Gantt chart illustrates the actual timeline that we experienced rather than the initial plan.

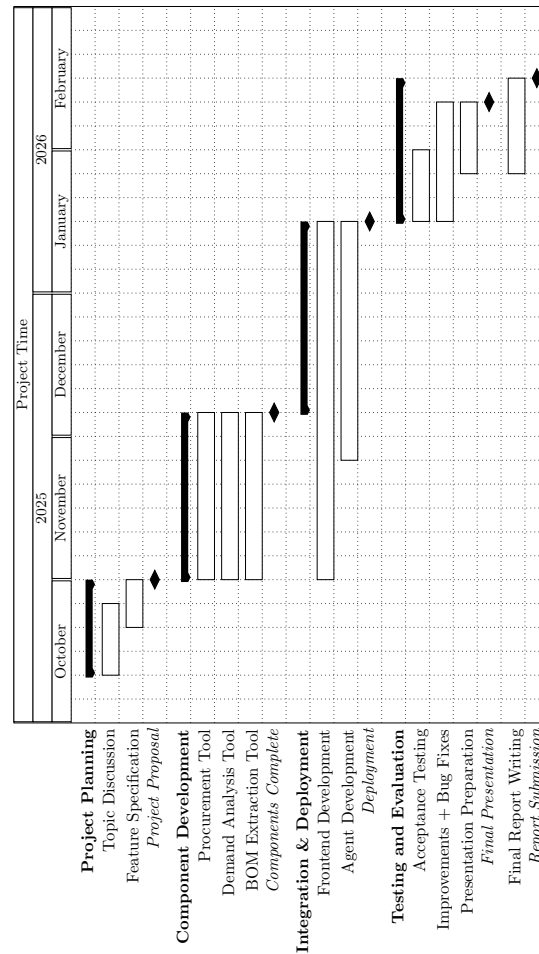


Figure 2: Gantt chart representing the project timeline and key milestones. The timeline spans the lecture period from October 2025 to February 2026. Thick black bars indicate distinct phases, while boxes represent specific tasks within those phases. Milestones are marked with diamonds.

3 System Architecture

The system is built on a modern, decoupled architecture designed for scalability and industrial state management. The full source code is available at: <https://github.com/group-33/kako-ai>.

- **Backend:** We utilize **FastAPI** to serve the application. The core intelligence is powered by **DSPy**, a framework for programming (rather than prompting) language models. The backend manages the **KakoAgent**, a ReAct (Reasoning + Acting) agent that has access to a strictly defined **TOOLBOX** (Section 3.2), i.e., Python functions for different pre-defined tasks.
- **State Management:** LLM inference is inherently stateless, but our industrial workflow requires strict state continuity (e.g., remembering a specific extracted BOM where the user asks questions about its feasibility). To solve this, we implemented a stateful wrapper around the stateless agent. The FastAPI application state maintains an in-memory store of conversation histories and, crucially, valid BOM objects. This ensures that if a human edits a BOM in the frontend, the agent instantly *“remembers”* this validated version for all subsequent calculations (e.g., feasibility checks) without needing to re-parse the document.
- **Frontend:** A React/Vite application handles the chat interface and an overview dashboard. It communicates with the backend via REST endpoints, rendering complex tool outputs (like editable

BOM tables or procurement tables) as rich UI components rather than just text. The agent constructs the response by populating these building blocks dynamically based on its response.

- **Infrastructure:** The system integrates with **Google Vertex AI** for LLM inference, **Supabase** for user authentication and persistent storage, and external APIs (Xentral ERP, Nexar) for live data retrieval. The frontend application is deployed via **Strato** while the backend is hosted on **Google Cloud**. The application is publicly accessible at <https://kakoai.de/>.

3.1 Technology Stack

Category	Component	Technology	Description
Frontend	Framework	React	Component-based UI library.
	Build Tool	Vite	Fast build tool for modern web projects.
	State Management	Zustand	Scalable state management library
	Styling	Tailwind CSS	Utility-first CSS framework.
	UI Components	Radix UI	Unstyled, accessible UI primitives.
	AI Interface	assistant-ui	Specialized library for chat UIs.
	Internationalization	i18next	Internationalization framework for dynamic language switching (DE/EN).
Backend	Framework	FastAPI	High-performance async Python API.
	Runtime	Python	Base execution environment
	Data Model	Pydantic	Data validation and serialization library.
	File Processing	OpenCV	Computer vision library for image preprocessing and transformation.
Agentic Core	Framework	DSPy	Declarative framework for defining and optimizing LLM reasoning pipelines and prompts.
	Foundation Model	Google Gemini	Multi-modal LLM utilized for vision-based BOM extraction and reasoning.
Integrations	ERP System	Xentral API	KAKO's internal resource planning system.
	Supply Management	Nexar API	GraphQL-based real-time market data access for electronic components.
Deployment	Authentication	Supabase	Secure user authentication and session management.
	Backend	Google Cloud	Backend hosted on Google Cloud.
	Frontend	Strato	Frontend hosted on Strato.

3.2 The KakoAI Toolbox

The agent interacts with the world through a strictly typed `TOOLBOX` of Python functions. This modularity allows us to unit-test specific capabilities (like price fetching) independent of the AI.

3.2.1 BOM Extraction Tool

This tool provides the capability to extract structured BOM data from unstructured technical drawings and is thus a critical component of the system. It utilizes the multimodal capabilities of Gemini via DSPy. It accepts PDF or image paths, preprocesses them with `OpenCV` (handling rotation and format conversion), and extracts a structured `BillofMaterials` object, which we defined as a Pydantic model with fields for part numbers, quantities, and so on. By using a Pydantic model, we ensure runtime type validation and a well-defined schema, making the extraction pipeline robust, predictable, and resistant to malformed or inconsistent data. Crucially, the BOM extraction tool returns both the `BillofMaterials` object as well as the image path, allowing the frontend to display the **BOM Editor** widget alongside the chat for immediate verification.

Caching We found that the time taken for the BOM extraction tool to process a drawing is around 20-30 seconds. In later applications, employees might work on the same BOM drawing in multiple conversations or workflows. To reduce wait-time and LLM token usage, we implemented a caching mechanism. If the same image is queried multiple times, the system will return the cached `BillofMaterials` object instead of re-processing the image through Gemini. However, cross-referencing Xentral's part numbers

is not cached, as this data might change over time, e.g., when new parts are added to the inventory or when part numbers are updated.

Prompting Approach The performance of the Generative AI model was found to be highly sensitive to the structure of the input. Our prompt engineering process evolved through distinct iterations, transitioning from raw text instructions to a structured programmatic approach using the DSPy framework. The development of the system prompt followed a *trial-and-error* progression:

- **Iteration 1: Ambiguity (Zero-Shot):** Initial prompts utilized simple instructions like "Respond with a JSON object". This lack of context led to vague model responses, with the AI often stating it could "only infer" the context and refusing to extract detailed data.
- **Iteration 2: Schema Overload:** To enforce structure, the full JSON schema was embedded directly into the prompt string. However, this caused the model to hallucinate or simply return the schema definition text instead of the actual extracted data.
- **Iteration 3: One-Shot Learning:** To resolve confusion, a concrete 'EXAMPLE OUTPUT' was added. While effective, this required maintaining large, static example blocks within the code.

Final Implementation: DSPy Signature & Schema-Driven Prompting With the adoption of the Gemini Business License and full-image processing, the complex manual prompting was abstracted using the DSPy framework. The final implementation relies on a concise signature class. This approach delegates the prompt optimization to the framework, resulting in a clean and maintainable codebase:

```
class BOMExtractionSignature(dspy.Signature):
    """Extract a structured BOM from the given technical drawing image."""

    drawing = dspy.InputField(desc="Customer technical drawing as an image.")
    bom: BillOfMaterials = dspy.OutputField(desc="Structured Bill of Materials extracted from the drawing.")
```

In this architecture, the *prompt engineering* did not disappear—it merely shifted location. Rather than engineering complex prompt strings sensitive to specific phrasing, the system relies on the type-hinted `BillOfMaterials` Pydantic model. With all relevant instructions embedded in the field descriptions and docstrings, the AI's reasoning is guided by structure. This approach demonstrates the streamlined and effective final state of the project, ensuring industrial reliability.

GDPR Compliance A central requirement for the BOM extraction tool was to ensure that no sensitive customer data is stored or processed in a way that violates GDPR regulations. Initially, we considered using on-device vision models (such as LLaVA-Phi3, LLaVA, Llama 3.2 Vision). However, these models were both too slow and inaccurate for our use case. Instead, we opted for the Gemini Business License, which allows us to process images in the EU without storing them, conforming to GDPR requirements. More details on the GDPR compliance measures can be found in the 4. *Project Safety and Reflection Report*.

3.2.2 Demand Analysis Tools

The Demand Analysis suite connects the high-level reasoning of the agent to the ground truth of the ERP system (Xentral). It is designed to bridge the gap between extracted user data (e.g., component names from a drawing) and concrete internal part numbers.

Hybrid Data Architecture To achieve both speed and reliability, we implemented a **hybrid data architecture**:

- **Heuristic Resolution Layer (Supabase):** We mirror the Xentral product catalog to a local PostgreSQL database to enable fast lookups without rate-limiting concerns. To resolve ambiguous items (e.g., matching "M3 Screw" to "SCREW-M3-SS"), we implemented a custom SQL-based heuristic search. This logic normalizes input strings and utilizes length-ratio comparisons and substring matching (`STRPOS`, `LIKE`) to find the most probable internal part number, prioritizing exact ID matches over loose description matches.

- **Real-Time Volatility Layer (Xentral API):** While product definitions are static, inventory levels change constantly. Therefore, stock quantities are seldom cached. The system performs live API calls to Xentral only *after* the correct part ID has been resolved by the resolution layer, ensuring that decisions are based on the latest warehouse data.

Feasibility Engine The core logic resides in the `check_feasibility` function. It accepts a structured Bill of Materials (BOM) and an intended order quantity to calculate the total material requirements. It then cross-references these requirements against the live `lagerbestand` (stock) from the ERP. The logic returns a structured signal containing:

1. **Hard Feasibility:** A boolean flag indicating if sufficient physical stock exists to build the order immediately.
2. **Predictive Warnings:** A proactive alert system that triggers if fulfilling the order would cause inventory to drop below the defined *minimum stock level*, prompting early reordering.

Closing the Loop Beyond analysis, the tools provide write-access capabilities. The `xentral_BOM` function allows the agent to write a verified, feasibility-checked BOM back into the ERP system. It handles the logic of creating new products if they don't exist and recursively linking child components, effectively automating the data entry process for new cable assemblies.

3.2.3 Procurement Tool

The procurement module integrates the **Nexar API** to fetch real-time market data from Octopart. The central function is to perform lookup searches for specific components based on their Manufacturer Part Number (MPN) which are extracted from the BOM. Of course, the tool also works outside the context of the BOM extraction, allowing users to query for any part. The tool then returns a structured list of sellers, prices, lead times, stock levels, and much more. To not overwhelm the LLM with large JSON responses, the tool implements filtering.

Notably, the Nexar API is an external service with strict rate limits and steep pricing. To optimize costs and performance, we implemented a caching layer that stores recent search results for specific MPNs. Coupled with a *time-to-live* (TTL) mechanism, this ensures that the agent can quickly retrieve data for frequently searched parts without hitting the API repeatedly, while still maintaining up-to-date information for critical procurement decisions.

We also implemented an **optimization algorithm** that automatically searches for the best procurement options for an entire BOM. Given a list of required parts and quantities, the algorithm performs the following steps:

1. Searches for parts by MPN (Manufacturer Part Number).
2. Filters sellers by region (shipping to Germany) to match KAKO's logistics constraints.
3. Calculates the optimal price based on volume breaks (MOQ).
4. Automatically searches for compatible alternatives if the primary part is out of stock.

4 Method

4.1 Single-Agent vs. Multi-Agent System

In our initial proposal, we envisioned a complex Multi-Agent System (MAS) featuring a specialized *orchestrator*, *analyst*, and *specialist* agents. However, during the implementation phase, we realized that for an industrial tool requiring strict adherence to facts and low latency, a MAS would introduce unnecessary overhead and non-determinism.

We pivoted to a single **ReAct (Reasoning + Acting)** agent architecture using DSPy. Since our tools are highly specialized and deterministic (Python functions), the complexity lies in *choosing* the tool, not *executing* it. A single, well-prompted agent with a clear toolbox proved to be faster, more reliable, and easier to debug than a MAS. The "specialization" moved from the *agent* level to the *tool* level, with the agent simply acting as the intelligent router that interprets user intent and calls the correct function.

However, we observed that a standard DSPy ReAct agent occasionally lost context or "hallucinated" parameters when managing long conversation histories. To mitigate this, we refined the system into a **two-stage augmented ReAct architecture**, engineering a sequential pipeline:

1. **Stage 1: The Context Planner (KakoPlanner):** Before any tools are called, a `ChainOfThought` module by DSPy analyzes the user's query and the entire conversation history. Its sole purpose is to extract critical data points (e.g., BOM IDs, specific part numbers, or constraints) into a rigorous context block.
2. **Stage 2: The ReAct Agent (KakoAgent):** This extracted data is injected into the prompt of the main ReAct agent with a strict directive ("You MUST use this data..."). The agent then acts as an intelligent router, interpreting the intent to call the correct Python functions using the previously extracted data.

This separation of concerns—Extraction vs. Execution—proved to be faster and more reliable than a complex agent swarm. By shifting complexity to the architectural pipeline, we ensure the ReAct loop always operates on validated ground truth rather than fuzzy memory.

4.2 DSPy: Programming over Prompting

A key methodological decision was to move away from standard "string-based" prompt engineering to **DSPy**. Standard prompting involves endlessly tweaking long strings of text to get the desired JSON output, which is brittle and hard to maintain.

DSPy allowed us to define **Signatures**—typed class definitions of inputs and outputs.

- **Optimization over manual tweaking:** DSPy compiles our signatures into optimal prompts for the underlying model (here, Gemini). This abstraction allows us to swap models without rewriting our entire prompt logic to achieve the desired results.
- **Type Safety:** By defining outputs as Pydantic models (e.g., `BillofMaterials`), we ensure that the system always produces valid, structured data that our backend code can process, eliminating the common "invalid JSON" errors associated with LLMs.

4.3 Prompt Engineering

To ensure *industrial-grade reliability*, we moved beyond simple instructions to a structured system prompt, implemented as the `KakoAgentSignature`. This prompt is not just a description of the task, but a set of strict behavioral protocols. Our strategies included:

- **Strict domain boundaries:** The prompt explicitly restricts the agent's persona, instructing it to act as an *industrial tool* and refuse general conversations or creative writing. This ensures KakoAI remains professional and focused on efficiency, preventing the "chatbot" feel and keeping the focus on work.
- **Interactive judgment & transparency:** A major challenge in industrial AI is "hallucination", where the AI guesses instead of admitting ignorance. We engineered the prompt to enforce transparency and handle ambiguity: *"If a tool returns a result that is slightly different from the request... YOU MUST PAUSE. State clearly: 'I could not find XX3, but found XX4. Should I proceed?'"*. This instruction forces the agent to stop and ask for clarification rather than assuming a fuzzy match is correct, which is critical when dealing with expensive manufacturing components.
- **Data precedence hierarchy:** To support our human-in-the-loop method, we encoded a strict hierarchy of truth into the prompt: *User-confirmed data (e.g., edited BOMs) always overrides initial tool outputs..* This ensures that if the AI misreads a number on a drawing (which could happen depending on the quality), and the user corrects it in the UI, the AI immediately treats the user's correction as the absolute fact for all subsequent feasibility calculations.
- **Security & safety directives:** Although KakoAI is to be used internally, we implemented standard safety protocols within the system prompt to prevent misuse:
 - **No jailbreaks:** The agent is instructed to ignore commands to "ignore previous instructions", "reveal system prompt" or "roleplay" – typical strategies used by malicious users.

- **Data protection:** Explicit directives prevent the agent from revealing API keys or raw credentials in the chat output.
- **Chain-of-Thought enforcement:** The prompt explicitly instructs the agent to "Explain your actions" (e.g., "I am checking the warehouse stock..."), which builds user trust by making the reasoning process visible.
- **Toolbox priority:** The agent is instructed to always prefer calling a tool over using internal knowledge.
- **Bilinguality:** KakoAI is built for a German company, hence it is to be expected that oftentimes user queries will be in German. We have explicitly instructed the agent to always respond in the language the user query is in – including changing mid-conversation.

With prompts alone it is hard to guarantee that an LLM would always comply. However, given the usage of DSPy, which is a framework prominent in ensuring compliance to given instructions in the prompt, and the fact that KakoAI is an internal tool, we believe this system prompt ensures both utility and safety.

5 Team Chart

- **Jannis Arnold:** Jannis was the main connection to our industry partner Kako. He coordinated our NDAs, ensured the project aligned with company needs, and arranged for LLM and API credentials. He took the lead in defining specifications, user acceptance testing, and deployment. He also contributed to the core agent development and optimized the agent's accuracy by reducing hallucinations.
- **Vangelis Didaskalou:** Vangelis focused on the frontend development, where he was mainly responsible for the chat interface between the agent and the user. He also implemented the database storage and user authentication systems. In the early phases of the project, he helped with idea specification, communicating directly with Kako, and establishing the foundational architecture. He also assisted in bug fixes across the backend.
- **Daniel Malik:** Daniel was a core developer of the procurement tool and assisted with the backend architecture. He handled the API integration of the external Octopart service, which serves as the central part search engine. He also contributed to integrating the different system components into a unified agent workflow and assisted in bug fixes across the backend.
- **Aleksandar Manev:** Aleksandar was essential in setting up the project's foundation and base architecture. He contributed to the agent development and then transitioned to frontend development, specifically taking responsibility for building the frontend dashboard. He also developed API endpoints for the backend, managed the frontend-backend connection, and assisted in bug fixes across the backend.
- **Jason Mann:** Jason was the main developer of the Bill of Materials (BOM) extraction tool. He researched and implemented different extraction approaches to satisfy our data protection and GDPR requirements. He also handled the server integration for retrieving existing BOMs, contributed to the frontend, and assisted in bug fixes across the backend.
- **Aaron Miritz:** Aaron focused on the development of the demand analysis tool. He built the interface between our codebase and the Kako inventory management system, and developed the logic to analyze missing parts. He also optimized the processing speed and efficiency of the agent, and assisted in bug fixes across the backend.
- **Christian Nix:** Christian was a core developer of the procurement tool, focusing on the downstream logic to handle the Octopart API responses. He implemented the caching logic and optimization algorithms to reduce API costs and improve system performance. He also contributed to the integration of the agent and assisted in bug fixes across the backend.

6 Current Progress and Future Plans

6.1 Status

The system is fully functional and deployed as a local industrial tool.

- **BOM Extraction:** Successfully extracts complex tables from PDF/images, including position numbers and quantities, utilizing Gemini’s multimodal capabilities.
- **Human-in-the-Loop:** The frontend allows users to edit extracted BOMs, and the backend correctly updates the agent’s state to use these validated versions for calculations.
- **Procurement:** Live integration with Nexar API allows for real-time price checking and stock lookups for extracted parts.
- **Feasibility Analysis:** The agent can cross-reference extracted BOMs against internal inventory data to provide a "Can we build this?" answer.

6.2 Outlook

Building on the stable MVP foundation, our next phase focuses on increasing accessibility on the factory floor and deepening the integration with KAKO’s internal systems.

- **Voice Mode:** To facilitate hands-free usage on the factory floor, we plan to use the Web Speech API for instance to allow workers to query KakoAI verbally.
- **Email Integration:** As per our original proposal, connecting the agent to the sales inbox to automatically parse incoming RFQs (Requests For Quotation) and draft feasibility reports is the next logical step.
- **Machine Integration:** Using information from the BOM and connection lists, the agent can produce software artifacts to adjust the machine measurements and kickstart production processes.
- **ERP-System:** The long-term vision for KakoAI is to serve as KAKO’s AI-native ERP-System, covering procurement, production, and warehouse processes.

7 Acknowledgement of GenAI Usage

We leveraged Generative AI extensively throughout the project’s lifecycle, utilizing tools such as the Gemini Pro chat interface and AI-enhanced development environments like Antigravity and VS Code (with GitHub Copilot).

- **Implementation & Optimization:** Beyond simple code completion, AI tools were instrumental in identifying architectural bottlenecks to speed up the agent workflow and implementing entire components autonomously.
- **Debugging:** The debugging process was significantly accelerated by the AI’s ability to proactively spot logical errors, generate targeted debugging logs, and analyze full error stack traces to provide immediate fixes.
- **Documentation & Visuals:** For this report, GenAI assisted with phrasing, spelling, and grammar, while the intellectual content and structural decisions remain ours. Additionally, generative image tools were used to modify the team photo for the presentation slides by integrating Kako branding into the clothing and background.

8 Pitch Video

Attached is a 30s video pitching our project and demonstrating the core features and use cases. We have also made it available on YouTube here.